To Retain or to Adapt? Generalizing Continual Learning

Lanzillotta et al. (2026)

Presented by Jaleel Radhu

Motivation

1. **Real-time adaptation, not just retention.** Should a continual learner always try to remember everything?

   What should be the central objective of a Continual Learning agent? Retention or Adaptation?

2. **Proposes a general framework for designing CL algorithms: Predictive CL.**

   What does our choice of CL objective assume about the future?
   Training on all past data assumes the future will look like the past (average of the past).

   Making this assumption explicit turns the choice of objective into a design space for new CL algorithms.

Continual Learning Setup

Environment	$\mathcal{E} = (A,\ O,\ \mathcal{P})$	Action space A, observation space O, and a fixed rule $\mathscr{P}$ for generating the next observation.
History	$H_n = \{(a_0, o_1),\ (a_1, o_2),\ \ldots,\ (a_{n-1}, o_n)\}$	Everything the agent has seen up to step n. It grows by one pair every step.
Learn	$\pi_n \sim \mathcal{A}(\cdot \mid H_n)$	The learning algorithm turns the history into a policy. This is where learning happens.
Predict	$a_n \sim \pi_n(\cdot \mid H_n)$	The current policy makes a prediction (action).
Observe	$o_{n+1} \sim \mathcal{P}_n := \mathcal{P}(\cdot \mid H_n,\ a_n)$	The environment produces new data. $\mathscr{P}$ is fixed, but $\mathscr{P}_n$ changes over time because it depends on the growing history: this is the non-stationarity.
Evaluate	$\ell(\pi_n,\ o_{n+1}) \geq 0$	Error of the current policy on the next observation, before it has trained on it: the real-time error.

Continual Learning Objective

Given this setup what should our CL objective be?

Say we have a series of steps: step 1, step 2, . . . step k, . . . , N

1. To retain our training memory of all the task seen so far?

   OR

2. To minimize our average error on the all the steps from 1 to N?

Continual Learning Objective

Given this setup what should our CL objective be?

Say we have a series of steps: step 1, step 2, . . . step k, . . . , N

1. To retain our training memory of all the task seen so far?

OR

2. To minimize our average error on the all the tasks from 1 to N?

Are 1 and 2 the same thing?

Continual Learning Objective

Average lifelong error (ALE):

$$\text{Inf A } \mathrm{ALE}(\mathcal{A}) := \frac{1}{T}\sum_{n=1}^{T} \mathbb{E}_{\pi_n \sim \mathcal{A}(\cdot|H_n),\, o_{n+1}\sim\mathcal{P}_n}\left[\ell(\pi_n, o_{n+1})\right].$$

subject to computational constraints

We want to the average lifelong error of the agent to be minimized.

Continual Learning Objective

Average lifelong error (ALE):

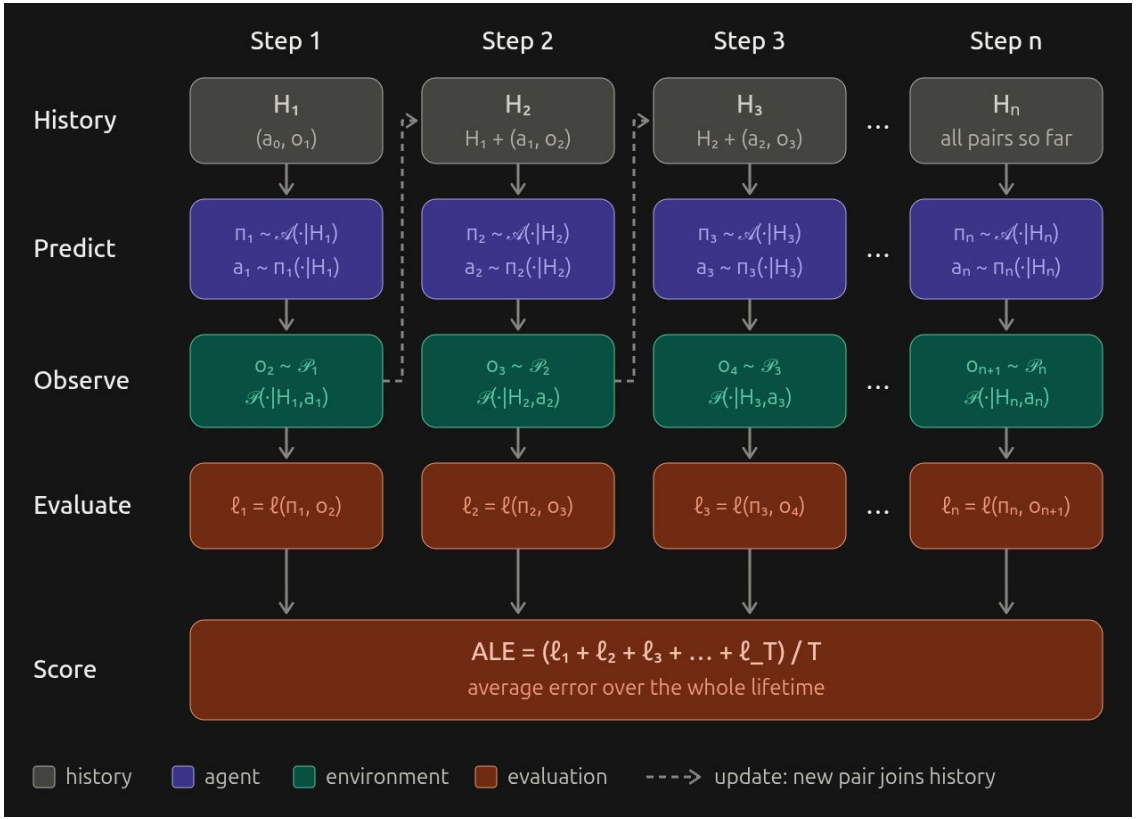
$$\text{Inf A } \mathrm{ALE}(\mathcal{A}) := \frac{1}{T}\sum_{n=1}^{T}\mathbb{E}_{\pi_n\sim\mathcal{A}(\cdot|H_n),\, o_{n+1}\sim\mathcal{P}_n}\left[\ell(\pi_n, o_{n+1})\right].$$

subject to computational constraints

We want to the average lifelong error of the agent to be minimized.

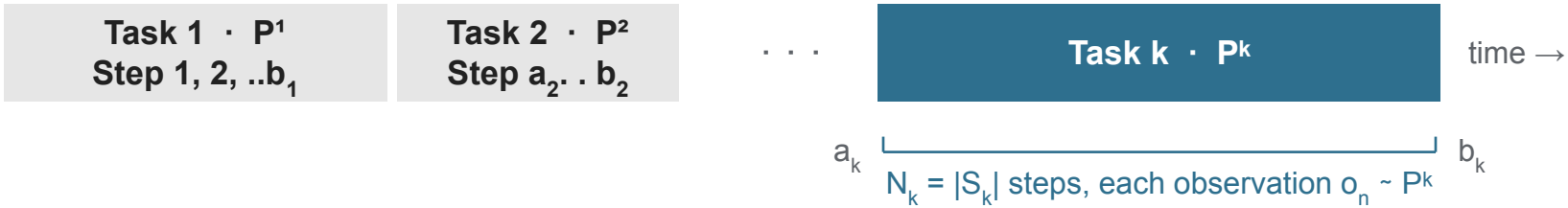
We need to find A, that is the learning algorithm or the algorithm which updates/trains the policy/model, which minimizes ALE.

Continual learning objective



Task-Based Setup

Each task k is an interval $S_k = [a_k, b_k]$ where data come from one fixed distribution P^k. The data generating process P_n is locally stationary.



Average task error

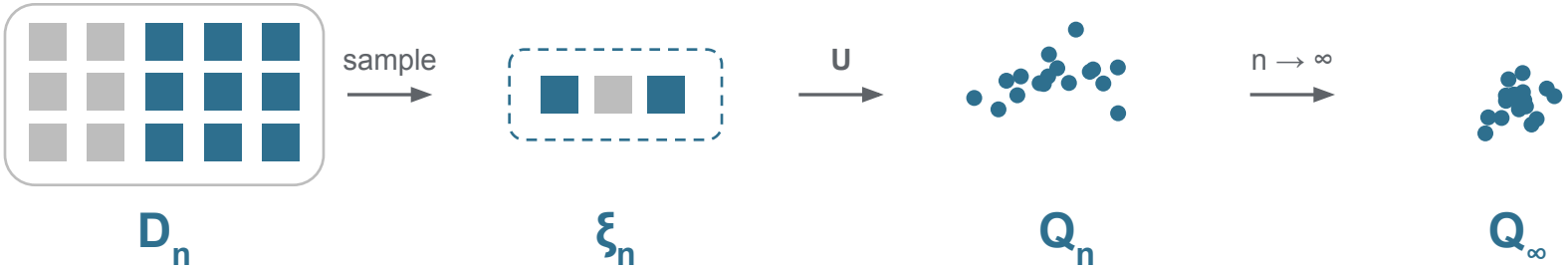
$$\mathrm{ALE}^k(\mathcal{A}) = \frac{1}{N_k} \sum_{n=a_k}^{b_k} \mathbb{E}_{\theta \sim \mathcal{A}(\cdot|H_n)}[\mathcal{L}^k(\theta)]$$

Real-time error during task k, with $\theta_n \sim A(\cdot|H_n)$

$ALE(A) = \Sigma_k (N_k / T) \cdot ALE_k(A)$

Lifelong error is a weighted average of task errors: a task counts in proportion to how long it lasts.

The learning Algorithm



Sampling distribution

Picks the training data; replay mixes in past tasks (gray).

Data batch

The examples drawn from D_n for one update step.

Parameter distribution

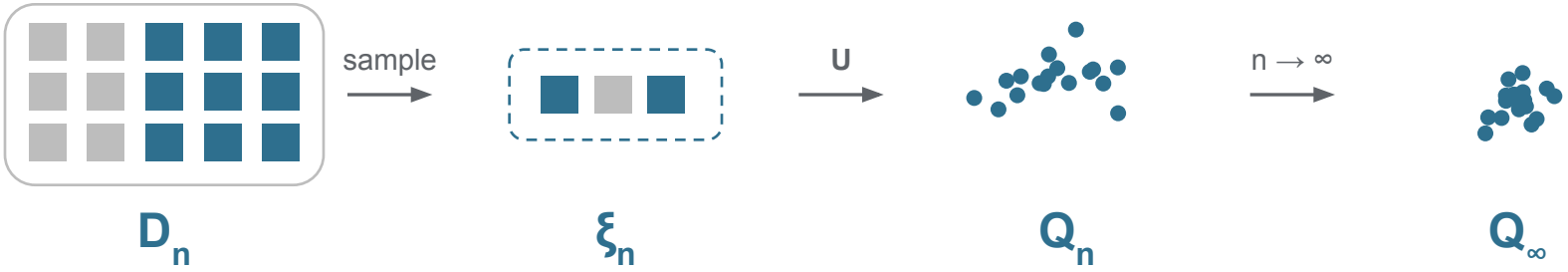
Spread of θ_n across random seeds at step n.

Stationary limit

Where Q_n settles if training stays on one task.

One training step: $\theta_{n+1} = U(\theta_n, \xi_n), \quad \xi_n \sim D_n$ **(e.g. one SGD step)**

The learning Algorithm



Sampling distribution

Picks the training data; replay mixes in past tasks (gray).

Data batch

The examples drawn from D_n for one update step.

Parameter distribution

Spread of θ_n across random seeds at step n.

Stationary limit

Where Q_n settles if training stays on one task.

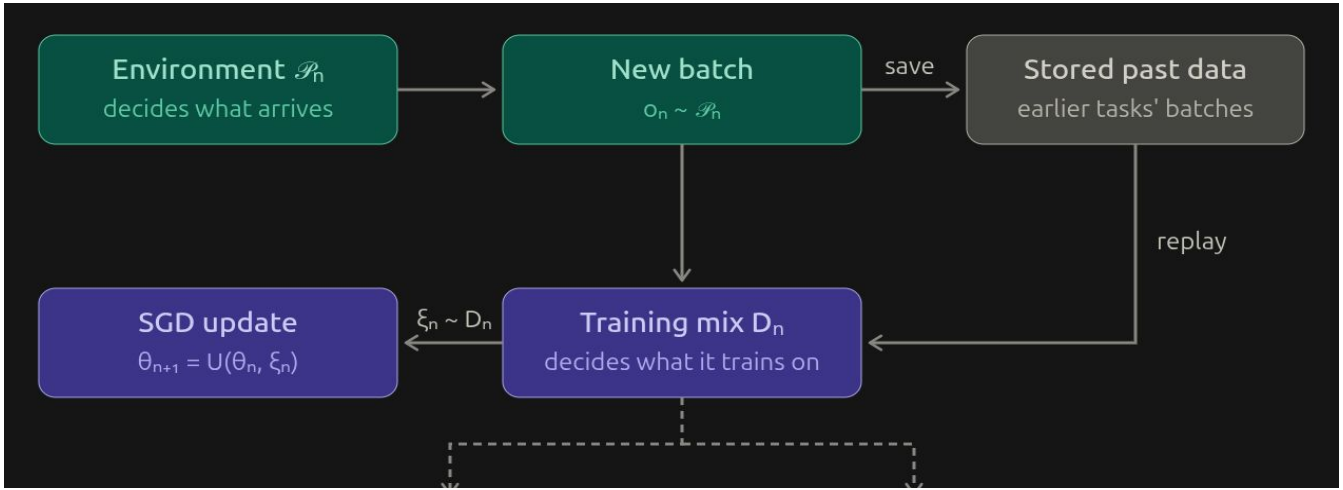
One training step: $\theta_{n+1} = U(\theta_n, \xi_n), \quad \xi_n \sim D_n$ **(e.g. one SGD step)**

Q_0: distribution of the starting parameters θ_0 (the random initialisation)

Q_1: distribution of θ_1, after one update step on a random batch ξ_0

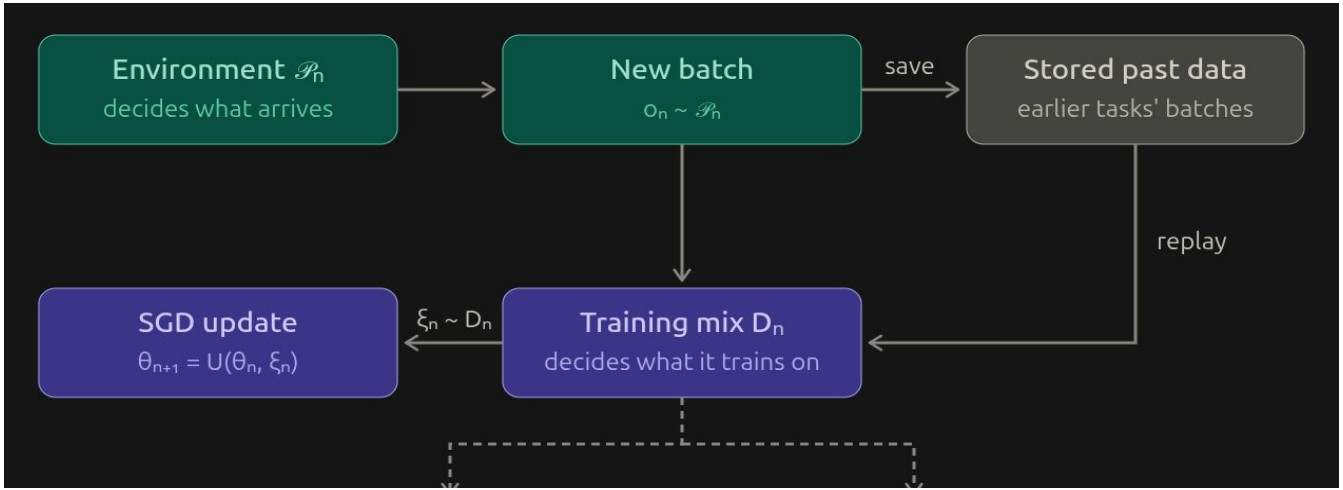
The learning Algorithm

P_n vs D_n: the environment's **P_n decides what data arrives;** the algorithm's **D_n decides what it trains** on.
Within a task, $D_n = D_k$.and $P_n = P_k$
(Note: n is the time step number and k is the task number.



How to choose D_k?

- Only have control over our choice of D_k, that is what distribution/data we choose to train our model on after each time step withing a Task k, AND
- U the learning algorithm. (e.g SGD, Adam, etc)



How to choose D_k?

- One option is we store all data seen in all the tasks seen so far and train on all of it.
- This is one extreme

How to choose D_k?

- One option is we store all data seen in all the tasks seen so far and train on all of it.
- This is one extreme

This gives us the joint task learning (**JTL) algorithm** where:

$$\mathcal{D}_{JTL}^{k} = \sum_{j=1}^{k} w_j^k \cdot \mathcal{P}^j, \qquad \text{where } w_j^k = \frac{N_j}{b_k}.$$

W^k_j is just the fraction of task j's duration in all the tasks from 1 to k.

Just to recall: N_j = Duration of Task j; b_k= is the final time step of task k

How to choose D_k?

- The other option is we forget everything seen and trained on so far. And start with a fresh model before each task k.
- This is the other extreme

How to choose D_k?

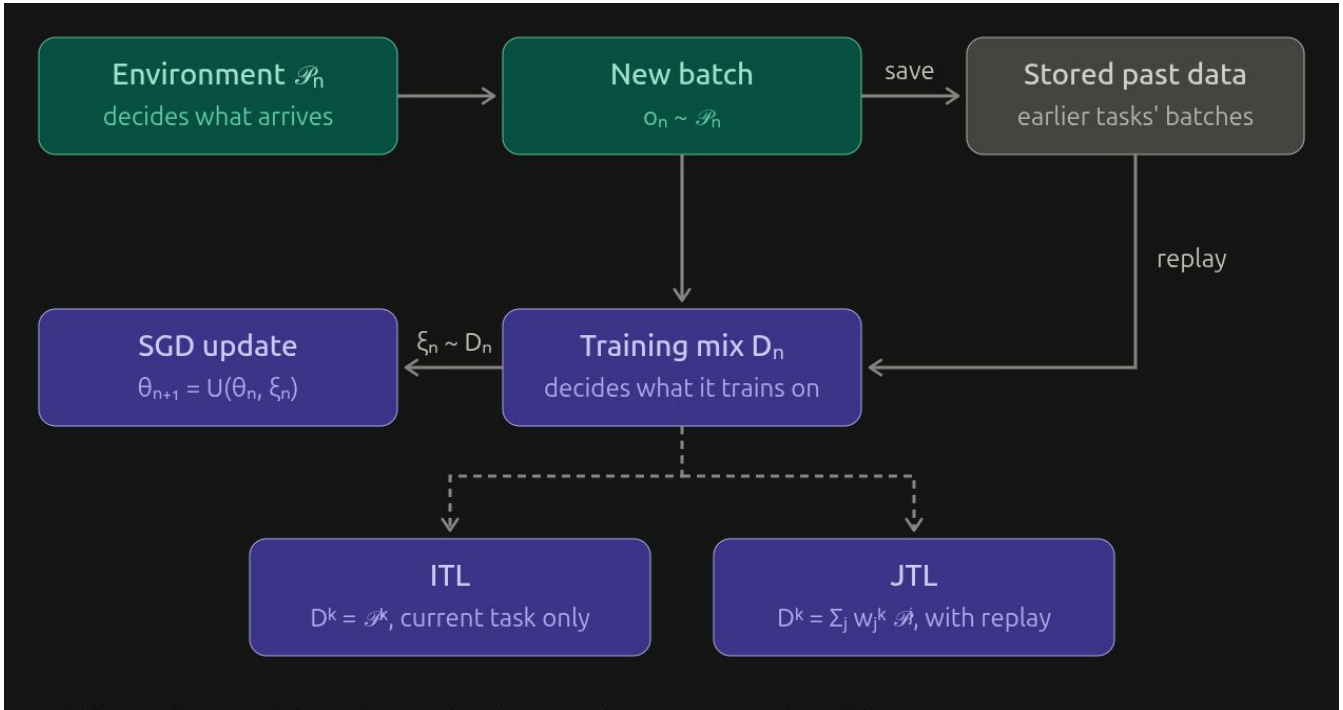
- The other option is we forget everything seen and trained on so far. And start with a fresh model before each task k.
- This is the other extreme

This gives us the Independent task learning (ITL) algorithm:

$$\mathcal{D}_{ITL}^{k} = \mathcal{P}^{k} .$$

JTL vs ITL

- We get two extremes of CL algorithms in the current setup



JTL vs ITL

- Which is better: JTL or ITL?
- Put another way: *does memory help*?

JTL vs ITL

- Which is better: JTL or ITL?
- Put another way: *does memory help*?

- To answer this, the paper defines Transfer Efficiency (TE):

$$\mathrm{TE}^k := \mathrm{ALE}^k(\mathcal{A}_{\mathrm{ITL}}) - \mathrm{ALE}^k(\mathcal{A}_{\mathrm{JTL}}).$$

Gain of JTL over ITL on task k (ALE^k = average error on task k)

JTL vs ITL

- Which is better: JTL or ITL?
- Put another way: *does memory help*?

- To answer this, the paper defines Transfer Efficiency (TE):

$$\mathrm{TE}^k := \mathrm{ALE}^k(\mathcal{A}_{\mathrm{ITL}}) - \mathrm{ALE}^k(\mathcal{A}_{\mathrm{JTL}}).$$

Gain of JTL over ITL on task k (ALE^k = average error on task k)

TE > 0: memory helps. Memory of past tasks lower the error on task k.

JTL vs ITL

- Which is better: JTL or ITL?
- Put another way: *does memory help*?

- To answer this, the paper defines Transfer Efficiency (TE):

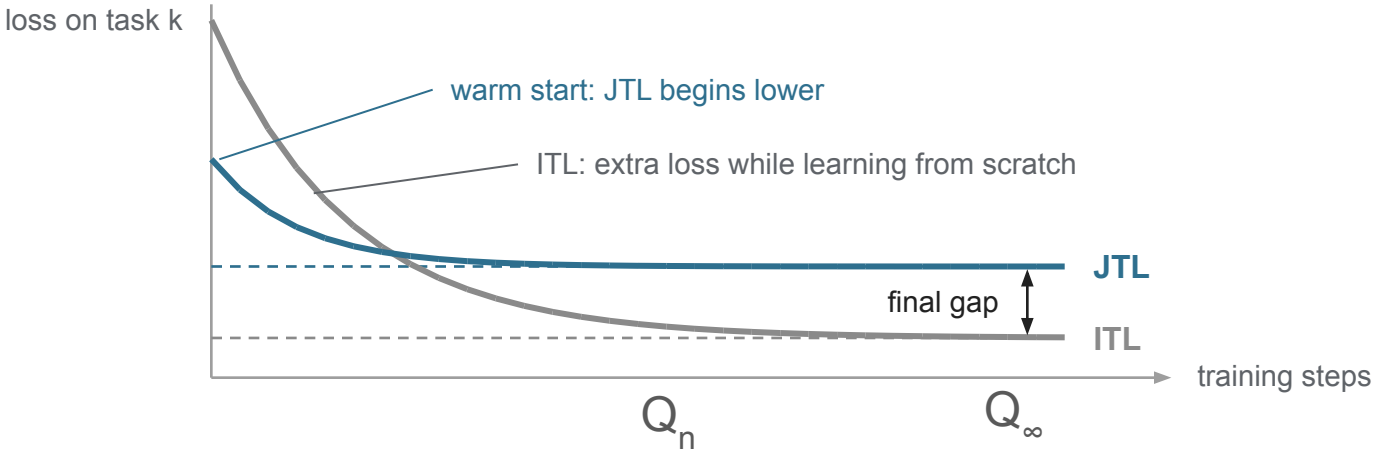
$$\mathrm{TE}^k := \mathrm{ALE}^k(\mathcal{A}_{\mathrm{ITL}}) - \mathrm{ALE}^k(\mathcal{A}_{\mathrm{JTL}}).$$

Gain of JTL over ITL on task k (ALE^k = average error on task k)

TE < 0: memory hurts. Past tasks raise the error on task k.

Instability

- Why can TE be positive or negative?
- Follow the loss on task k as each agent trains:



JTL starts lower (warm start), but can settle higher.

ITL starts from scratch, but settles at task k's own best.

Instability

- Where does each agent end up?
- Compare their losses once training has converged:

- Instability (I): JTL's extra loss at convergence

$$I^k := L^k(Q_\infty^{\mathrm{JTL}}) - L^k(Q_\infty^{\mathrm{ITL}})$$

I > 0: past tasks leave JTL at a worse final solution

L^k: expected loss on task k; $\mathcal{L}^k(Q) := \mathbb{E}_{\theta\sim Q}[\mathcal{L}^k(\theta)]$

Q_∞: parameter distribution after training converges.

Assumption: The training will converge.

Transient Error

- How fast does each agent get there?
- Measure the loss it carries above its final level:

- Transient Error (δ): average extra loss (over the stationary loss) while learning

$$\delta^k(\mathcal{A}) = \frac{1}{N_k} \sum_{n \in S_k} \left[L^k(Q_n^{\mathcal{A}}) - L^k(Q_\infty^{\mathcal{A}}) \right]$$

Small δ: the agent converges quickly. A warm start lowers δ.

Transient Error

- How fast does each agent get there?
- Measure the loss it carries above its final level:

$$\delta^k(\mathcal{A}) = \frac{1}{N_k} \sum_{n \in S_k} \left[L^k(Q_n^{\mathcal{A}}) - L^k(Q_\infty^{\mathcal{A}}) \right]$$

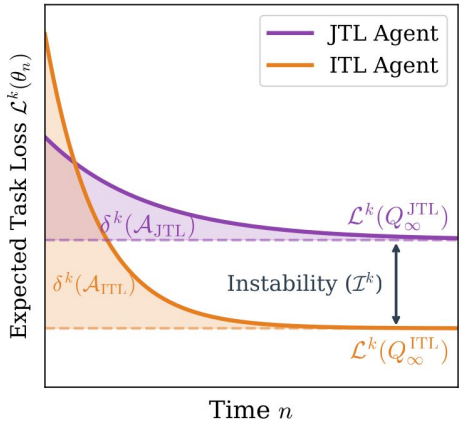


Figure is from the paper

Small δ: the agent converges quickly. A warm start lowers δ.

Instability and Transient Error

- Does memory help? Weigh both effects:
- TE splits exactly into the two terms:

$$\mathrm{TE}^k = -I^k + \left(\delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})\right)$$

−I: memory can worsen the final solution (stationary gap).

The gap you get even after getting best possible model in both ITL and JTL process

δ gap: memory can speed up learning (learning-speed gap) [

Instability and Transient Error

- Does memory help? Weigh both effects:
- TE splits exactly into the two terms:

$$\mathrm{TE}^k = -I^k + \left(\delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})\right)$$

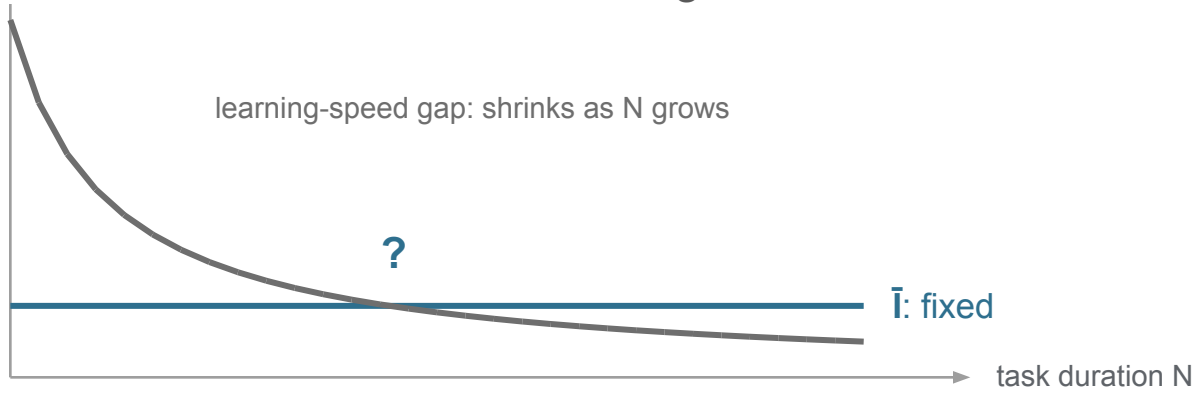
−I: memory can worsen the final solution (stationary gap).

The gap you get even after getting best possible model in both ITL and JTL process

δ gap: memory can speed up learning (learning-speed gap) [increasing n

The Critical Task Duration

- What happens as tasks get longer?
- Track both terms of TE as the task duration N grows:



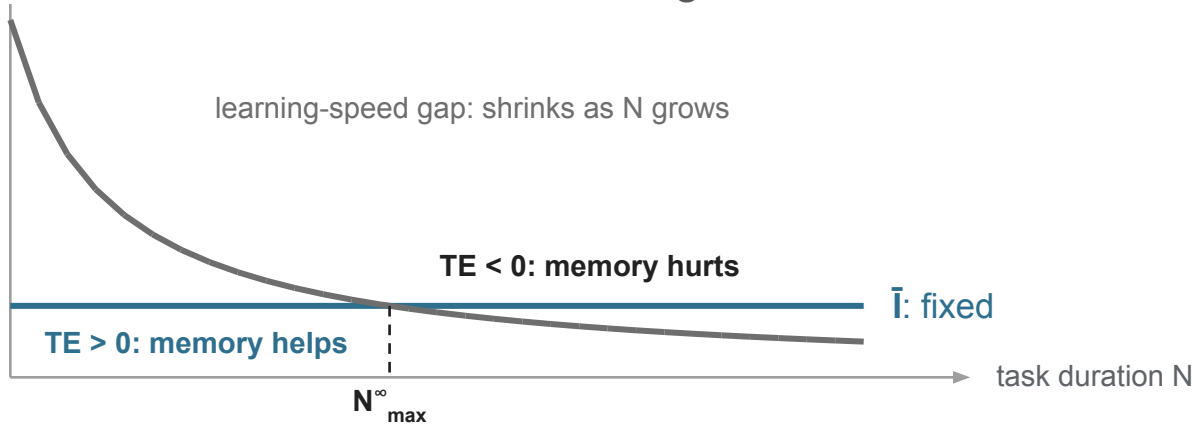
Instability Ī does not depend on N: it stays fixed.

The learning-speed gap shrinks as N grows.

Assumption: The training will converge.

The Critical Task Duration

- What happens as tasks get longer?
- Track both terms of TE as the task duration N grows:



Past the crossing, ITL catches up and TE turns negative.

That crossing is the Critical Task Duration, N^{∞}_{max}.

Assumption: The training will converge.

Critical Task Duration: New Terms

- Setup: K tasks, each lasting N steps
- Cumulative transient error: total extra loss over task k

$$\Delta^k(\mathcal{A}) = N\delta^k(\mathcal{A}) = \sum_{n \in S_k} [L^k(Q_n^{\mathcal{A}}) - L^k(Q_\infty^{\mathcal{A}})]$$

- Average both quantities over the K tasks:

$$\bar{\Delta}(\mathcal{A}) = \frac{1}{K}\sum_{k=1}^{K}\Delta^k(\mathcal{A}), \qquad \bar{I} = \frac{1}{K}\sum_{k=1}^{K} I^k$$

- Sublinear: f(N) $\in$ o(N) means f(N) / N $\to$ 0 as N $\to \infty$

  e.g. $\sqrt{N}$ and log N are sublinear; N / 2 is not

Critical Task Duration: Assumptions

- Convergence: the cumulative transient error Δ is sublinear, $\Delta \in o(N)$
- Positive instability: average Instability $\bar{I} > 0$
- Then transfer becomes harmful (TE < 0) beyond:

$$N_{\max}^{\infty} = \frac{\bar{\Delta}(\mathcal{A}_{\mathrm{ITL}}) - \bar{\Delta}(\mathcal{A}_{\mathrm{JTL}})}{\bar{I}}$$

$\Delta \in o(N)$: the extra loss per step, Δ / N, dies out

Larger $\bar{I}$ (a less predictable world): shorter window for transfer

Critical Task Duration: Proof (1/2)

- Step 1: start from the definition of TE and split it (Lemma 5):

$$\mathrm{TE}^k = \mathrm{ALE}^k(\mathcal{A}_{\mathrm{ITL}}) - \mathrm{ALE}^k(\mathcal{A}_{\mathrm{JTL}}) = -I^k + \delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})$$

- Step 2: average over the K tasks; TE > 0 exactly when

$$\frac{1}{K}\sum_{k=1}^{K}\left(\delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})\right) > \bar{I}$$

Left: average learning-speed gap. Right: average Instability.

Critical Task Duration: Proof (2/2)

- Step 3: substitute $\delta^k = \Delta^k / N$ into Step 2:

$$\frac{\bar{\Delta}(\mathcal{A}_{\mathrm{ITL}}) - \bar{\Delta}(\mathcal{A}_{\mathrm{JTL}})}{N} > \bar{I}$$

- Step 4: $\bar{I}$ is fixed, but the left side $\to 0$ as $\Delta \in o(N)$; they meet at

$$N^{\infty}_{\max} = \frac{\bar{\Delta}(\mathcal{A}_{\mathrm{ITL}}) - \bar{\Delta}(\mathcal{A}_{\mathrm{JTL}})}{\bar{I}}$$

So TE < 0 once $N > N^{\infty}_{\max}$: ITL ends up ahead

Empirical Validation: Benchmarks

- Three benchmarks, chosen to span a range of Instabilities:

CLEAR

Image classification

10 image classes evolving smoothly over 2004–2014

Smooth drift: low Instability

MD5

Image classification

5 unrelated public datasets in sequence, with sharp semantic jumps

Sharp shifts: high Instability

MT10

Reinforcement learning

10 Meta-World robot-manipulation tasks, known for task interference

Interfering tasks: high Instability

Empirical Validation: Setup

- Same task duration N for all tasks
- Hyperparameters tuned per algorithm for the lowest ALE
- ITL and JTL run with matched compute
- Score: Average Lifelong Accuracy or Success rate (ALA / ALS)
- TE is signed so that TE > 0 means JTL beats ITL

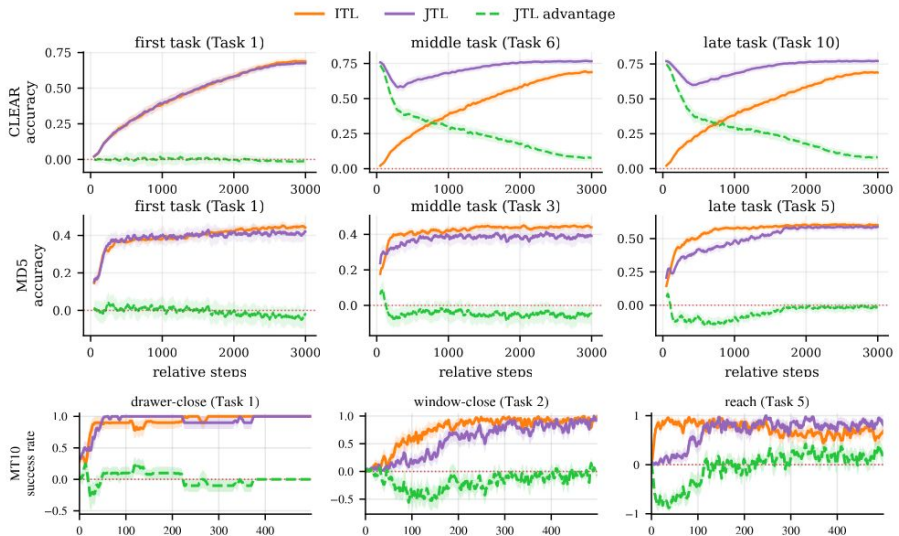


Figure is from the paper

Empirical Validation: Setup

- Same task duration N for all tasks
- Hyperparameters tuned per algorithm for the lowest ALE
- ITL and JTL run with matched compute
- Score: Average Lifelong Accuracy or Success rate (ALA / ALS)
- TE is signed so that TE > 0 means JTL beats ITL

Results in the wild (higher is better; MT10 reports success rate)

Benchmark	N	ITL	JTL	TE
CLEAR	3,000	0.465	0.681	+0.216
MD5	3,000	0.470	0.430	−0.040
MT10	500	0.514	0.274	−0.240

Empirical Validation: Predictions

- Both predictions follow from the TE decomposition:

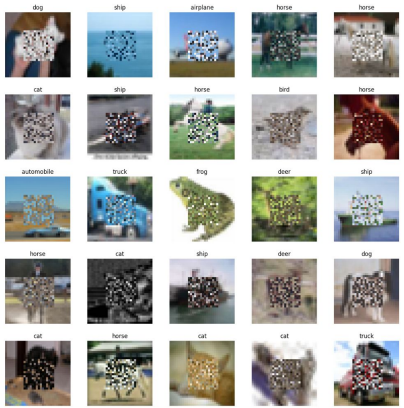
$$\mathrm{TE}^k = -I^k + \left(\delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})\right)$$

- P1 : Instability shifts TE by a constant, whatever N is

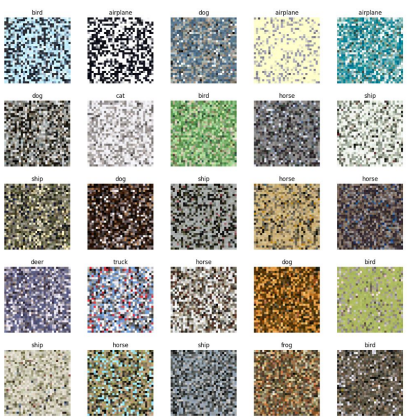
  TE then decays toward that constant at a 1/N rate

Testing P1: Controlling Instability on CIFAR-10

- Permuted (PC-16/24/32): pixels shuffled inside a centred box
- Shuffled (SC-10/30/50): 10/30/50% of labels reassigned
- Expect Shuffled to be more unstable: one image, conflicting labels



PC-16: 16×16 box (Fig. 18)



PC-32: 32×32 box (Fig. 19)

Figure is from the paper

Testing P1: Results

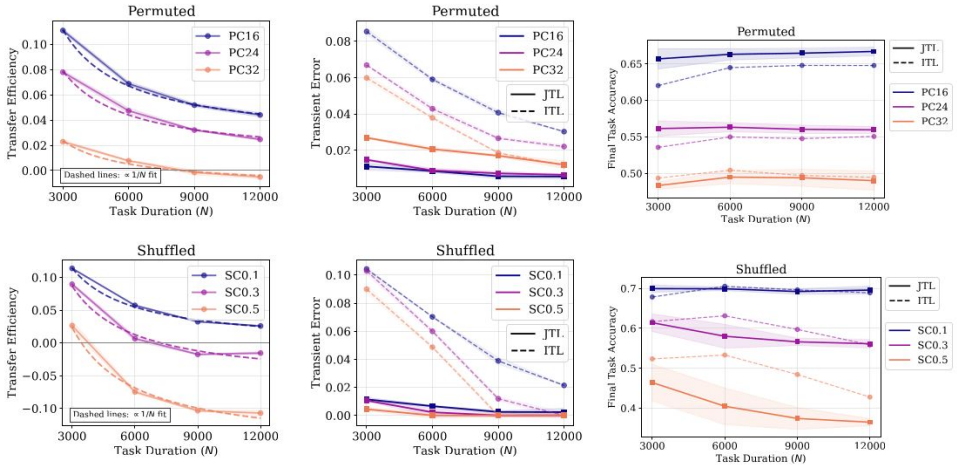


Figure 5: **Manipulating Instability on CIFAR-10.** Permuted-CIFAR (*Top*) and Shuffled-CIFAR (*Bottom*) across task durations N. *Left:* Transfer Efficiency decays as $1/N$ (dashed fits); higher Instability translates the curves vertically downward. *Middle:* JTL Transient (solid) is negligible relative to ITL Transient (dashed) at every N, matching the asymptotic prediction. *Right:* Final task accuracy ceilings. For interpretability, all quantities are measured in *accuracy* (between 0 and 1).

Figure is from the paper

- Left: stronger shifts move the TE curves down, and TE decays as 1/N (dashed fits)
- Middle: JTL transient error ≈ 0, so TE is driven by Instability and ITL's transient

Empirical Validation: Predictions

- Both predictions follow from the TE decomposition:

$$\mathrm{TE}^k = -I^k + \left(\delta^k(\mathcal{A}_{\mathrm{ITL}}) - \delta^k(\mathcal{A}_{\mathrm{JTL}})\right)$$

- P2 f: if $\bar{I} > 0$, a longer N turns TE from positive to negative

  Once both agents converge, TE is dominated by −I

Testing P2: Critical Task Duration on CLEAR

Sweep N ∈ {3,000, 6,000, 9,000, 12,000} steps per task on CLEAR

N	ALA ITL	ALA JTL	TE	δ ITL	δ JTL	I conv
3,000	0.465	0.681	+0.216	0.227	0.079	−0.066
6,000	0.562	0.713	+0.151	0.195	0.063	−0.011
9,000	0.610	0.720	+0.110	0.166	0.057	−0.003
12,000	0.641	0.732	+0.100	0.148	0.054	−0.0006

- TE falls from +0.216 to +0.10 as N grows: ITL converges further, shrinking δ ITL and I conv
- CLEAR has $\bar{I} \approx 0$, so TE decays toward 0 instead of crossing it
- MD5 and MT10: TE is already negative at the smallest N tested

I conv: finite-time estimate of Instability. Accuracy in [0, 1]; ± over seeds omitted. Source: Table 2 of the paper.

Predictive CL: Is There an Optimal Objective?

- Neither JTL nor ITL minimizes ALE in every environment
  - Unpredictable (adversarial) world: no first-order method beats online gradient descent on the current task
  - Partly predictable world: an algorithm can do better (Rakhlin & Sridharan, 2013)
  - If the next distribution were known, we would sample training data from it and minimize the next-step loss directly

Every Objective Encodes a Model of the Future

- Choosing what to train on is an implicit bet on what comes next:

JTL

Future = past

The world is ergodic: the classical i.i.d. premise of empirical risk minimization. Replay and anti-forgetting methods approximate it.

ITL

No usable structure

Nothing can be reliably extrapolated, so online gradient descent on the current task is minimax-optimal.

Predictive CL

Explicit future model

Make the model of the future a design choice instead of leaving it implicit in the objective.

Predictive CL (Definition 9)

- A prediction mapping Φ turns the history into a predicted distribution for the current task

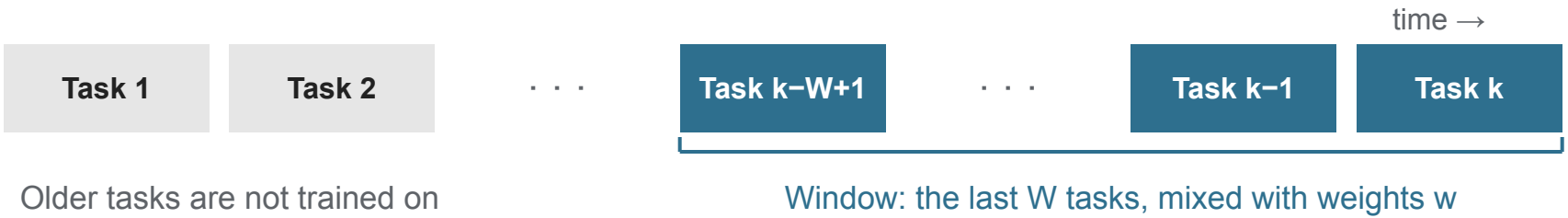
$$\inf_{\mathcal{A}} \frac{1}{N_k} \sum_{n=a_k}^{b_k} \mathbb{E}_{\theta_n \sim \mathcal{A}(\cdot|H_n)} \mathbb{E}_{o \sim \tilde{P}^k} [\ell(\theta_n, o)], \qquad \tilde{P}^k = \Phi(H_{\leq k})$$

- JTL: Φ predicts a weighted mix of all past tasks
- ITL: Φ predicts the current task only
- JTL and ITL are special cases, so the best Φ for an environment matches or beats both

Proof of Concept: the Window Algorithm

- Predict the future from the last W tasks and train on a mix of them:

$$D_W^k = \sum_{j=k-W+1}^{k} w_j^k \, P^j$$



Window Size Interpolates Between ITL and JTL

- The Window agent never resets its parameters, like JTL



ITL

Train on the current task only, starting from the initialized parameters

Window

Train on recent W tasks only

JTL

Train on every past task: an infinite window

Better Future Models

The Window encodes one prior: recent tasks predict the next, summarized by a single number W. The same framework allows richer choices of Φ:

Generative model

Produces samples that resemble future tasks

Selective replay

A learned replay distribution that does not sample the past uniformly

Thank You !